

Sometimes the
smallest step in the
right direction
ends up being the
biggest step
of your life



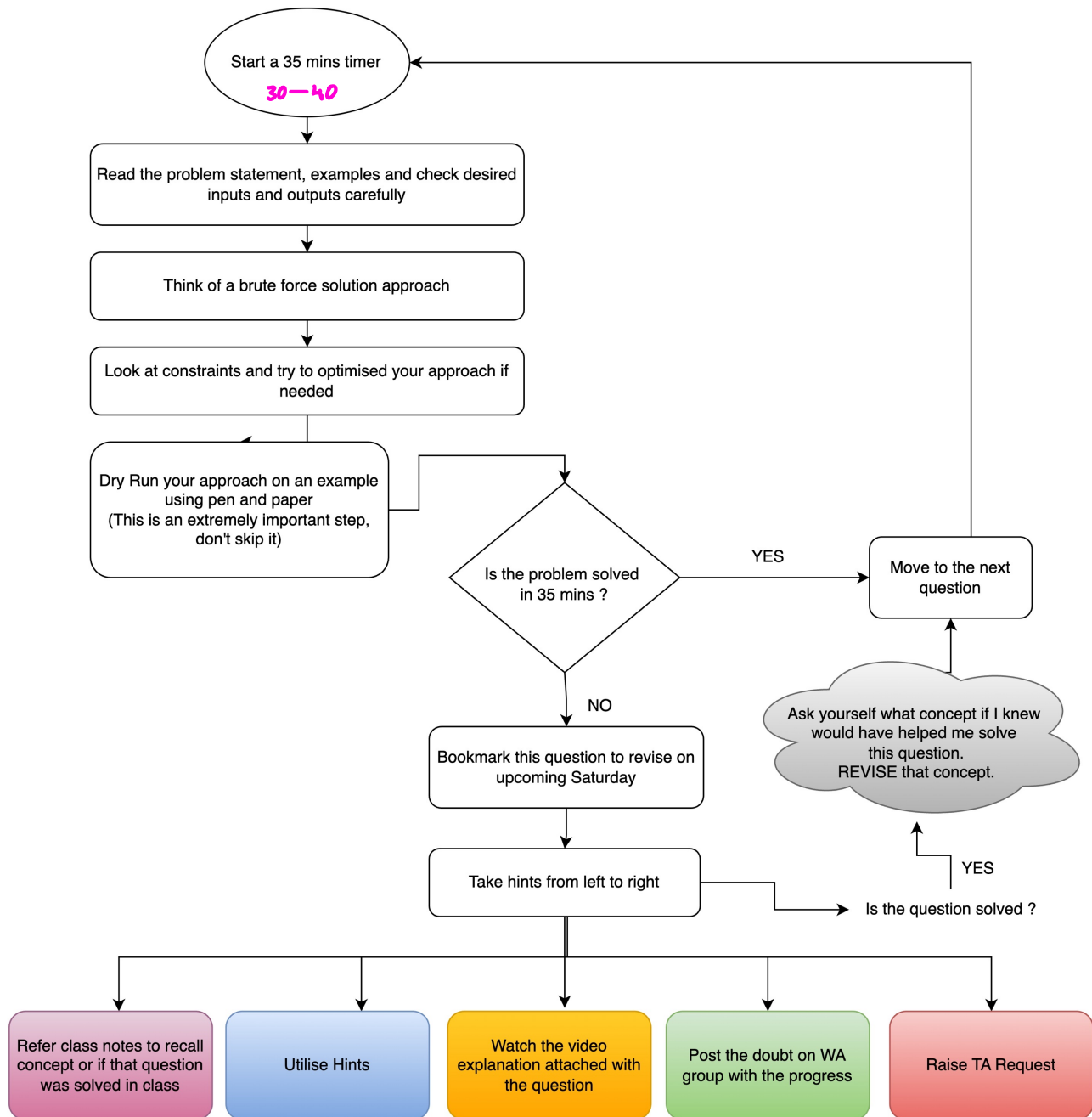
- Naeem Callaway

Agenda:

1. Max Subarray Sum (Kadane's Algorithm)
2. Continuous Sum Query from i to last index
3. Continuous Sum Query from i to j
4. Merge Overlapping Intervals



How to approach a problem ?





< Question > : Given arr[N]. Find the maximum subarray sum out of all subarrays.

continuous part of $A[i]$ $1 \leq N \leq 10^5$

Example 1 : arr[] $\rightarrow$ [-3 , 2 , 4 , -1 , 3 , -4 , 3]

Ans = 8

Example 2 : arr[] $\rightarrow$ [4 , 5 , 2 , 1 , 6]

Ans = 18

$$\text{Ans} = \sum_{\forall i} A[i]$$

Example 3 : arr[] $\rightarrow$ [-4 , -3 , -6 , -9 , -2]

Ans = -2

$$\text{Ans} = \forall i \max(A[i])$$

Brute force $\rightarrow$ $\forall$ subarray calculate sum &

$$\frac{N * (N+1)}{2}$$

take max as answer.

$$TC = O(N^3)$$

$$SC = O(1)$$

iterating



```

ans = A[0];
for (i -> 0 to N - 1) { // start to N
    for (j -> i to N - 1) { // end i—j
        for (k -> i to j) {
            sum += A[k];
        }
        ans = Math.max(ans, sum);
        sum = 0; // Reset sum for the next iteration
    }
}
return ans;

```

→ subarray sum

Prefix Sum → subarray sum $i—j = P[j] - P[i-1]$

$P[0] = A[0]$

```

for i → 1 to (N-1) {
    P[i] = P[i-1] + A[i]
}

```

$ans = A[0]$

```

for i → 0 to (N-1) {
    for j → i to (N-1) { // i—j
        if (i == 0) sum = P[j]
        else sum = P[j] - P[i-1]
        ans = max(ans, sum)
    }
}

```

}

$$TC = O(N^2)$$

$$SC = O(N) \rightarrow O(1)$$

return ans

use A[] for prefix sum.

Carry Forward

$$A = \begin{matrix} & 0 & 1 & 2 \\ [& 2 & 5 & 3 &] \end{matrix}$$

$$2 \rightarrow 2$$

$$ans = A[0]$$

$$2 \ 5 \rightarrow 7$$

for $i \rightarrow 0$ to $(N-1)$ {

$$2 \ 5 \ 3 \rightarrow 10$$

$$sum = 0$$

$$5 \rightarrow 5$$

for $j \rightarrow i$ to $(N-1)$ { // $i-j$

$$5 \ 3 \rightarrow 8$$

$$sum += A[j] \text{ // calculate}$$

$$3 \rightarrow 3$$

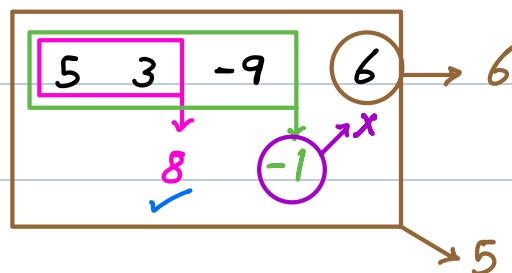
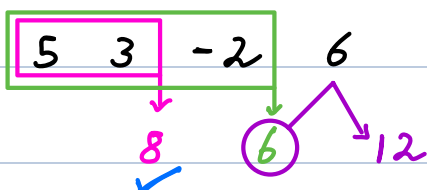
$$ans = \max(ans, sum) \text{ // use}$$

}

} return ans

$$TC = O(N^2)$$

$$SC = O(1)$$



Kadane's Algo.



arr[] → [5 , 6 , 7 , -3 , 2 , -10 , -12 , 8 , 12 , -4 , 7 , -2]

sum → ~~5~~ ~~11~~ ~~18~~ ~~15~~ ~~17~~ ~~7~~ ~~-5~~ ~~8~~ ~~20~~ ~~16~~ ~~23~~ ~~21~~

ans → ~~5~~ ~~11~~ ~~18~~ ~~20~~ 23

0 1 2 3 4 5 6
A = [-2 3 4 -1 5 -10 7]

sum = ~~-2~~ ~~3~~ ~~7~~ ~~6~~ ~~11~~ ~~1~~ ~~8~~

ans = ~~-2~~ ~~3~~ ~~7~~ 11

ans = A[0]

sum = 0

for i → 0 to (N-1) {

sum += A[i]

if (sum > ans) ans = sum

if (sum < 0) sum = 0

}

return ans

TC = O(N) SC = O(1)

~~i~~ ~~i~~ ~~i~~ ~~i~~
0 1 2 3
A = [-3 -5 -2 -8]

sum = ~~0~~ ~~-3~~ ~~-8~~ ~~-10~~ ~~-18~~ ~~-26~~

ans = ~~-3~~ -2

HW → Find subarray with max sum.

[start end]



Continuous Sum Query

< Question > : Initially all elements of an $arr[N]$ are 0. Then you are given Q queries. Every query contains i -idx and value. Increment elements from i 'th idx to last idx by the given value. Return final state of $arr[]$.

$$1 \leq N \leq 10^5$$

$$1 \leq Q \leq 10^5$$

$arr[] \rightarrow [0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0]$

Queries $\rightarrow 3$

3 3 7 7 7 7
5 5 5

idx val

3 4 ✓ $[0 \quad 3 \quad 3 \quad 7 \quad 5 \quad 5 \quad 5]$

1 3 ✓

4 -2 ✓

$A = [0 \quad 0 \quad 0 \quad 0 \quad 0]$

1 3 ✓

2 5 5 5 5
6

0 2 ✓

4 1 ✓ $[2 \quad 5 \quad 5 \quad 5 \quad 6]$

Bruteforce $\rightarrow \forall$ query, travel from i to end & add v .

$$TC = O(N * Q) \quad SC = O(1)$$



arr[] $\rightarrow$ [0 0 0 0 0 0 0]

Queries $\rightarrow$ 3

+4 $\longrightarrow$

+3 $\longrightarrow$

-2 $\longrightarrow$

idx val

3 4 ✓

[0 3 0 4 -2 0 0]

1 3 ✓

Prefix sum

4 -2 ✓

[0 3 3 7 5 5 5]

for $i \rightarrow 0$ to $(Q-1)$ {

idx = B[i][0] val = B[i][1]

a[idx] += val

}

for $i \rightarrow 1$ to $(N-1)$ {

a[i] = a[i-1] + a[i]

}

TC = $O(Q + N)$

SC = $O(1)$



< **Question** > : Initially all elements of an `arr[N]` are 0. Given Q queries.

Every query contains `[ s, e, val ]`. Increment elements from s to e by val.

Return the final state of `arr[ ]`.

$$1 \leq N \leq 10^5$$

$$1 \leq Q \leq 10^5$$

arr[10] → [0 , ~~0~~ , ~~0~~ , ~~0~~ , ~~0~~ , ~~0~~ , ~~0~~ , ~~0~~ , ~~0~~ , ~~0~~]

Queries → 3

[0 4 ~~-3~~ 4 4 4 4 4 ~~-3~~ 4 4]

s e val

3 6 3 ✓

2 7 -3 ✓

1 9 4 ✓

A = [~~0~~ ~~0~~ ~~0~~ ~~0~~ ~~0~~ ~~0~~ ~~0~~ ~~0~~]
-1 3 4 -4 3 -3 1 -3

1 4 3 ✓

0 5 -1 ✓

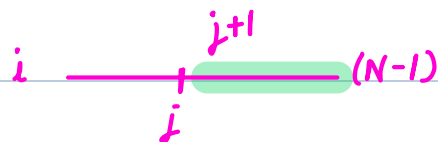
2 2 4 ✓

4 6 3 ✓

[-1 2 6 2 5 2 3 0]



$(i \ j \ v) \rightarrow (i \ v) \text{ and } (j+1) - v$
 $\downarrow$
 $\text{if } (j+1 < N)$



for $q \rightarrow 0$ to $(Q-1)$ {

$i = B[q][0] \quad j = B[q][1] \quad v = B[q][2]$

$a[i] += v$

if $(j+1 < N) \quad a[j+1] -= v$

}

for $i \rightarrow 1$ to $(N-1)$ {

$a[i] = a[i-1] + a[i]$

}

$TC = \underline{O(Q + N)}$

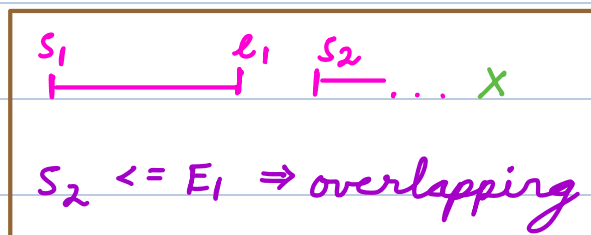
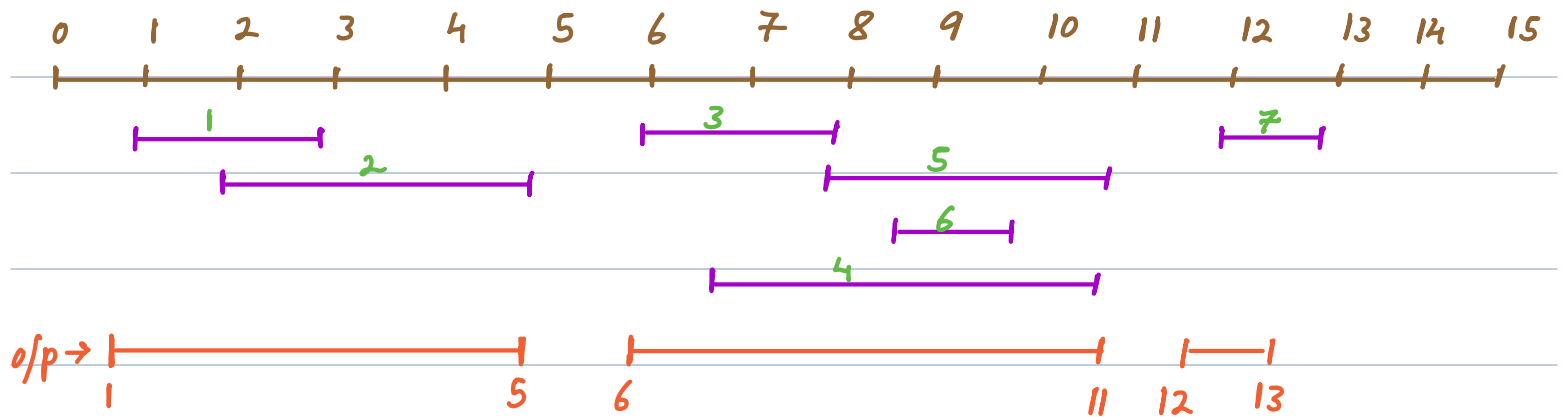
$SC = \underline{O(1)}$



Merge Overlapping Intervals

<Question>: You are given a collection of intervals A in a 2-D array format, where each interval is represented by a pair of integers [start, end]. The intervals are sorted based on their start values.

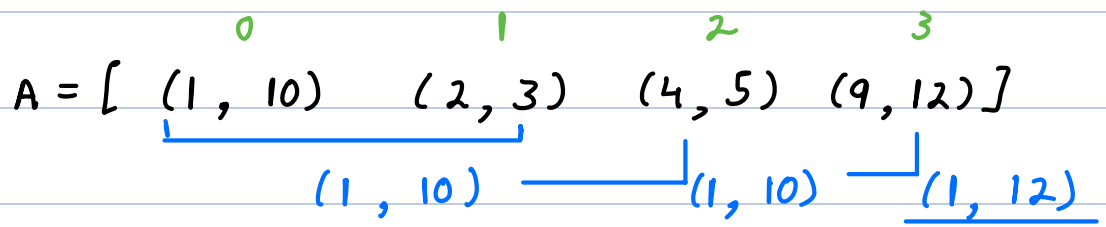
Your task is to merge all overlapping intervals and return the resulting set of non-overlapping intervals.



Merged interval $\rightarrow$

$$\text{start} = \min(s_1, s_2) = s_1$$

$$\text{end} = \max(e_1, e_2)$$



interval 1 & 3 are overlapping
but 2 is not overlapping X



$S = A[0][0]$ $E = A[0][1]$

for $i \rightarrow$ 1 to (N-1) {

$l = A[i][0]$ $r = A[i][1]$

 if ($l \leq E$) $E = \max(E, r)$

 else { print ($S + "$ ", " $+ E$)

$S = l$ $E = r$

 }

$TC = \underline{O(N)}$

} print ($S + "$ ", " $+ E$) ←

$SC = \underline{O(1)}$

⁰
 ^{~~i~~}
 ¹ ^{~~i~~}
 ²
 [(1, 2) , (4, 8) (6, 9)]

$o/p \rightarrow 1, 2$

$S = \cancel{1} 4$

$4, 9$

$E = \cancel{2} \cancel{8} 9$

